
Robby Hanovich

2 June 2026

Foundations of Python

GIT: <https://github.com/robbyhanovich/IntroToProg-Python-Mod07>

Assignment 07 – Classes and Objects.

Introduction

This Python program demonstrates file operations, using constants, variables, and print statements to display a message about a student's registration for a Python course. It uses dictionaries for data processing and applies various exception handling. The script also takes advantage of the JSON Python library.

This assignment demonstrates the use of Python Classes and Objects.

This script modifies the Assignment07-Starter.py script. The modifications center around implementing a Person Class and a Student Class that inherits the Person Class. The script also demonstrates the use of properties to get and set information, and the built-in methods of constructor `__int__()` and "to string" `__str__()`.

Data Classes vs Processing Classes

Classes are a way of grouping functions, variables, and constants by the name of the class. Grouping functions within classes create a modular structure, making it easier to manage and maintain code.

The associated methods (aka functions) within each class are listed to provide greater context for how the code is organized.ⁱ

Classes can be viewed as Processing Classes or Data Classes. Where the Processing class focuses on data processing or presentation and the Data Class focuses on organizing data.

Below is an example of a Processing class used within the script.

```
class FileProcessor: #used to read and write to file
    def read_data_from_file (file_name: str, student_data: list) -> list[dict[str,str]]:
    def write_data_to_file (file_name: str, student_data: list) -> None:
```

To call a method within a class use the format "class_name.method_name (parameter 1, ...)". For example, `FileProcessor.read_data_from_file (file_name, student_data)`, will read data from a file.

Data Classes are a way to represent information inside of a class so that you can access data unique to each class. The script defines a Person object and a Student object. Each of these objects contains unique data as well as a set of methods for getting, setting, and manipulating the data.ⁱⁱ

Below is an example of a Data Class used within the script.

```
class Person: # contains the first and last name of a person
    __first_name: str = ""
    __last_name: str = ""
```

The Student class inherits the class Person and adds information about the course the student is taking.

```
class Student (Person):
    __course_name: str = ""
```

Data Class Components

Data classes typically have Attributes, Constructors, Properties and Methods.ⁱⁱⁱ

Attributes:

An attribute is a piece of data, or a characteristic specific to an object. They can be different data types like strings, integers or custom data types. For example, the Person Class has attributes `first_name` and `last_name`.

Constructors:

A constructor is a special method (function) that is **automatically** called when an object of a class is created. Its primary purpose is to set an object's attributes values when it is created. In Python the constructor is named `__init__()`. Below are the constructors used to initialize the Person Class and the Student Class.

class Person:

```
def __init__(self, first_name: str = "", last_name: str = "") -> None:
    self.__first_name = first_name
    self.__last_name = last_name
```

class Student (Person):

```
def __init__(self, first_name: str = "", last_name: str = "", course_name: str = "") -> None:
    super().__init__(first_name = first_name, last_name = last_name) # call to Person Class constructor
    self.__course_name = course_name
```

Properties:

Properties are functions designed to manage attribute data. Typically, for each attribute, you create two properties—one for "getting" data and one for "setting" data. These functions are commonly known as "Getters" and "Setters". Error checking and data validation is usually done within the "Setter" property so that the error checking occurs each time the attribute within the class is set. Note, the "Getter" and "Setter" names much match.

The example below contains the "Getter" and "Setter" properties for the `first_name` attribute of the Person Class. The "Getter" and "Setter" properties for `last_name` and `course_name` follow the same pattern.

```
@property      #getter for first_name of Person class
def first_name(self) -> str:
    return self.__first_name.title() # return first name using title case

@first_name.setter # setter for first_name of Person class
def first_name(self, value) -> None:
    if str(value).isalpha():
        self.__first_name = str(value).title() # set the first name using title case
    else:
        raise Exception ("First names cannot contain numbers.\n")
```

Methods:

Python's "object" class includes several built-in methods. The two used in this script are `__int__()` constructor method and the `__str__()` "to string" method. The constructor method is discussed above, the `__str__()` method in Python is responsible for returning a human-readable string representation of an object. Defining a custom `__str__()` method enables you to control how your object is represented as a

string. The example below returns a string of the student's first name, last name, and course name as comma separated values.

```
def __str__(self) -> str:
    return f"{self.first_name}, {self.last_name}, {self.course_name}" # Vic, Vu, Python 100
```

Additional Script Notes.

The introduction of the Person and Student Data Classes into the script required modifying the code to read from and write to the "Enrollments.json" file since the student information is no longer stored in a .json friendly format.

The following snippet reads the "Enrollments.json" file into a json_students dictionary, loops through each dictionary entry, creates a new Student, and appends the new Student to the list of Student objects.

```
# Get a list of dictionary rows from the data file
file = open(file_name, "r")
json_students = json.load(file)

# Convert the list of dictionary rows into a list of Student objects
student_objects: list[Student] = []
for row in json_students:
    student_objects.append(Student(first_name = row['FirstName'], last_name = row['LastName'],
                                   course_name = row['CourseName']))
```

The following snippet converts the list of Student object into a dictionary and writes the data to the "Enrollments.json" file.

```
student_objects: list[dict[str, str]] = []
for student in student_data:
    student_objects.append ({ "FirstName": student.first_name, "LastName": student.last_name,
                              "CourseName": student.course_name })

file = open(file_name, "w")
json.dump ( student_objects, file, indent=2)
```

Summary

This script modifies the Assignment07-Starter.py script. The modifications center around implementing a Person Class and a Student Class that inherits the Person Class. The script also demonstrates the use of properties to get and set information, and the built-in methods of constructor `__int__()` and "to string" `__str__()`.

ⁱ Python Module 6 Notes, Randal Root, page 28

ⁱⁱ Python Module 7 Notes, Randal Root, page 5

ⁱⁱⁱ Python Module 7 Notes, Randal Root, page 6